CANDY
DIALOGUE ENGINE

Adding Commands

1.1.0

WARNING: Candy_Engine.gd is overwritten during updates. Always document any modifications you make to it, so that you can re-add them later.

New commands can be added in two ways to Candy Dialogue Engine: by creating entirely new commands and adding them to the **commands()** function, or by adding new options to the §Custom command.

Expanding the §Custom command is the simplest option: the command is already part of the Candy Dialogue Creator and therefore doesn't require any modifications to it. The downside is that all data for your custom command must be entered in a single text field, which isn't very practical for commands with many variables. Making use of Candy DC's Presets feature can be a big help, allowing you to create various pre-formatted versions of the §Custom command.

Creating an entirely new command, on the other hand, will require you to modify Candy DC to include it. This isn't very hard once you're familiar with the process, but it can be intimidating at first (see the "Adding Commands" guide in Candy DC for instructions). The benefit is that this allows you to fully tailor the command in the GUI.

1. §Custom Command

Open Candy_Engine.gd, scroll down to the Commands section, and to the bottom of the **commands()** function, where you'll find the §Custom command:

```
##########################################
#&                                      ##
#&              CUSTOM COMMANDS         ##
#&                                      ##
#? You may add your own custom commands here, either standalone or as part of the §Custom command.
#region - Custom Commands
"§custom":
        var command: Variant = resolve_value(command_value.get("Command", ""))
        var data: Variant = command_value.get("Data", "")

        if data.begins_with(candy_de.super_singleton_symbol):
                data = resolve_value(candy_de.singleton_symbol + data.substr(1))
```

```
	elif data.begins_with(candy_de.super_node_symbol):
		data = resolve_value(candy_de.node_symbol + data.substr(1))
	elif data.begins_with(candy_de.super_vardict_symbol):
		data = resolve_value(candy_de.vardict_symbol + data.substr(1))
	elif data.begins_with("sv_res://"):
		data = "v_res://" + resolve_value(data.substr("sv_res://".length()))
	elif data.begins_with("sv_user://"):
		data = "v_user://" + resolve_value(data.substr("sv_user://".length()))

	match command:
		#TODO: Add your own commands

		_:
			pass

#endregion - custom commands
```

The §Custom command accepts two pieces of data: "Command", which indicates the processing logic to use, and "Data", which is a string that can represent one or more values, similarly to arguments in a function.

If you want to provide a variable for the §Custom command's "Data" field, you'll need to use the 'super' variable symbols (default: ££, $$ and €€) when creating dialogues. When the §Custom command runs, this will replace the entire "Data" string with the variable's value.

Inside the 'match command:' statement, add a new case and give it a name that will remind you of what it's supposed to do. This will be the custom logic for your command.

Under that case, you'll need to write your own code to interpret the 'data' variable. Unless the data string contains only a single value, you'll probably want to convert it to a dictionary or an array if it's a string that represents multiple values.

If you want to include individual variables inside the "Data" field, to be converted to values individually at runtime, write them with the regular variable symbols (default: £, $ and €) when creating your dialogues. Then, under your command's 'match' case, call **resolve_value()** for each variable.

With 'data' now converted into proper values, you can continue by writing your custom command's code, to make it perform however you want.

The rest is up to you. If you need assistance, use the code of the built-in commands as templates or examples, or refer to the rest of the Candy DE documentation.

2. New Command

If you'd rather create new standalone commands, simply add a new match case to the 'match' statement that features all the other commands.

There isn't much to say, as this is open-ended and entirely dependent on what you want your new command to achieve.

Note that command names are normalized inside the **command()** function, using the **normalize_command_name()** function. This removes spaces and dashes (-) from command names, and converts them to lowercase. However, it preserves underscores (_). Therefore, make sure the cases you add to the 'match' statement don't contain uppercase letters, spaces or dashes.

For example, a command named **"§My_Custom Command-1"** will be normalized to **"§my_customcommand1"**.

We highly suggest using other commands as templates. This isn't mandatory, but they can be an excellent starting point and might feature code you can re-use.